



SCHEDULING

Allegra The Barman

Abstract

This assignment investigates the behavior of different CPU scheduling algorithms using a bar-scheduling simulation as an analogy for process scheduling. The algorithms evaluated were First-Come First-Served (FCFS), Shortest Job First (SJF), Priority Scheduling, and Multilevel Feedback Queue (MLFQ).

Tracey Letlape
ltltra001@myuct.ac.za

Table of Contents

Methods.....	2
Results	3
Conclusions.....	5
Statement on AI Usage	5
Formula Sheet.....	5
Formulas:	5

Methods

To generate data, I first defined my range of inputs as follows:

- My integer **seed** argument should be in the range of [1, 3],
- my **number of patrons** argument should be in the range of [8, 50], for which only three of the range are randomly selected as the experiment number of patrons values using the seed provided for the specific run,
- The **sched** argument is in the range of [0, 3], representing algorithms respectively.

I used a python script **run_experiments.py** to run different experiments using different experiment values and the **run_experiment.py** which runs a single experiment based on a certain combination.

The **recordCompletedOrder()** method in Barman.java records each completed drink order into experiment-specific output files. It creates the required result directories, generates a unique primary key for each order, calculates order-level timing metrics, and writes both raw order data and computed metrics to tab-separated files. The function is synchronised to prevent concurrent file-writing conflicts, clears stale files for the current run once per execution, and performs defensive checks to avoid duplicate primary keys across **OrderData** and **OrderMetrics**. This ensures that later analysis scripts can safely match each order's data with its corresponding metrics.

The following scripts were defined to help do the following:

1. **compute_patron_metrics.py:**

- The script converts order-level simulation output into patron-level summaries.
- Reads matching files from OrderData and OrderMetrics, links rows using the shared primary key, and groups all drink orders belonging to the same patron.
- It then produces two outputs: PatronData, which stores each patron's metadata; and PatronMetrics, which stores each patron's total waiting time, response time, turnaround time, and process turnaround time.
- It handles data validation by checking for duplicate primary keys to prevent corrupting experiment results.

2. **compute_stats_metrics.py:**

- Computes experiment run-level statistical metrics from the patron-level files.
- Outputs one StatsMetrics file per experiment run, which records the summary statistics like average, minimum, and maximum waiting time, response time, and turnaround time.
- It also computes other scheduling criteria metrics such as throughput ratio, fairness ratio, predictability ratio, and starvation possibility.

3. **plot_metric_graphs.py:**

- Handles the visualisation of all experiments results generated.
- Creates graph images from OrderMetrics, PatronMetrics, and StatsMetrics for each results file.
- To allow for clearly visible algorithm comparison graphs, also creates AlgoMetricsGraphs using selected statistics from all StatsMetrics files.

- The noPatrons_seed combinations are used as key to group algorithms together. With each algorithm represented by a bar.
4. **run_experiment.py:**
- Script can be imported by the batch runner (run_experiments.py) or executed directly from the command line.
 - Takes one combination of input values, clears the specific experiment previous outputs if exists, runs the Java simulation, compute patron metrics, computes statistical metrics, and generates the corresponding graphs.
 - Makes use of the structured Experiment and ExperimentResult data classes to keep experiment configuration and status information organised.
 - Log files are stored to store error messages in cases of failure.
5. **run_experiments.py:**
- This is the batch_experiment runner.
 - Builds a list of combinations as described at the beginning, calls the run_single_experiment() function from run_experiment.py
 - Calls the create_algo_metrics_graphs() after all individual experiment runs finishes, to generate the final cross-algorithm comparison graphs.

The layered structured of results and results was intended to mimic database structure, to simplify computations and allow for easily readable results.

To ensure the results were reproducible, the experiment I ran the experiments a multiple of times locally, storing the previous run results to a separate directory before I rerun it. Then compare the data results using the *BeyondCompare* software. The files were not exactly identical due to thread interleaving, and the values would slightly differ due to thread overhead, which is inevitable.

Results

After running the experiments, the below figure was derived. The results suggest the following:

As the number of patrons, the **average waiting time**, **response time**, and **turnaround time** for each algorithm increases. Which is expected, as theoretically, an increase in the number of processes means more processes competing for limited CPU service. The CPU gets busier, but individual processes spend longer waiting.

The Shortest Job First (SJF) scheduling algorithms performs best for the **average waiting time**, **response time**, and **turnaround time** metrics as compared to all other algorithms. This is expected as theoretically; the algorithm favours short burst times. The priority scheduling algorithm also performs well, falling slightly behind SJF and sometimes, even better. First-Come First-Serve (FCFS) is often worse than the above two as it suffers from the convoy effect. No favouritism, no prioritisations, patrons are served in the order in which they arrived. The Multi-Level Feedback Queue (MLFQ) algorithm performs worse than expected in most average time graphs, especially as the number of patrons increases.

For the **median values**, SJF again performs best, especially for median waiting and response time. The median helps show the typical patron experience without being overly affected by extreme outliers. MLFQ has high median values in larger experiments, meaning its poor average performance is not only cause by some patrons with very long average waiting times, but a typical patron also waits longer.

The **standard deviation** graphs show the spread of the results. Priority has a lower standard deviation, meaning the algorithm is more consistent and predictable. It has relatively lower standard deviation compared to SJF and MLFQ, suggesting more stable behaviour. SJF has good average and median performance, but its standard deviation increases in larger workloads, meaning more patrons wait much longer than others.

For **fairness**, FCFS and Priority generally perform well, while SJF often has lower fairness. This is expected, as FCFS serves jobs in arrival order, but SJF favours shortest jobs. MLFQ is not consistent, sometimes performs well, sometimes not.

For **predictability**, FCFS and Priority generally perform well. Predictability is linked to how much waiting times vary between patrons. Since SJF can create large differences between short and long jobs, it is less predictable. MLFQ is also inconsistent, which is not expected as theoretically it is expected to improve responsiveness by moving jobs between queues and using priority levels.

For starvation, almost all algorithms show very low starvation risk. Only SJF shows a spike in starvation ratio for the experiment with 12 number of patrons, using seed = 1. This is theoretically expected because the SJF's favouritism behaviour can starve long jobs if shorter jobs keep arriving. Priority scheduling can also cause starvation in theory, especially if low-priority jobs are repeatedly delayed, but these experiment results suggest otherwise.

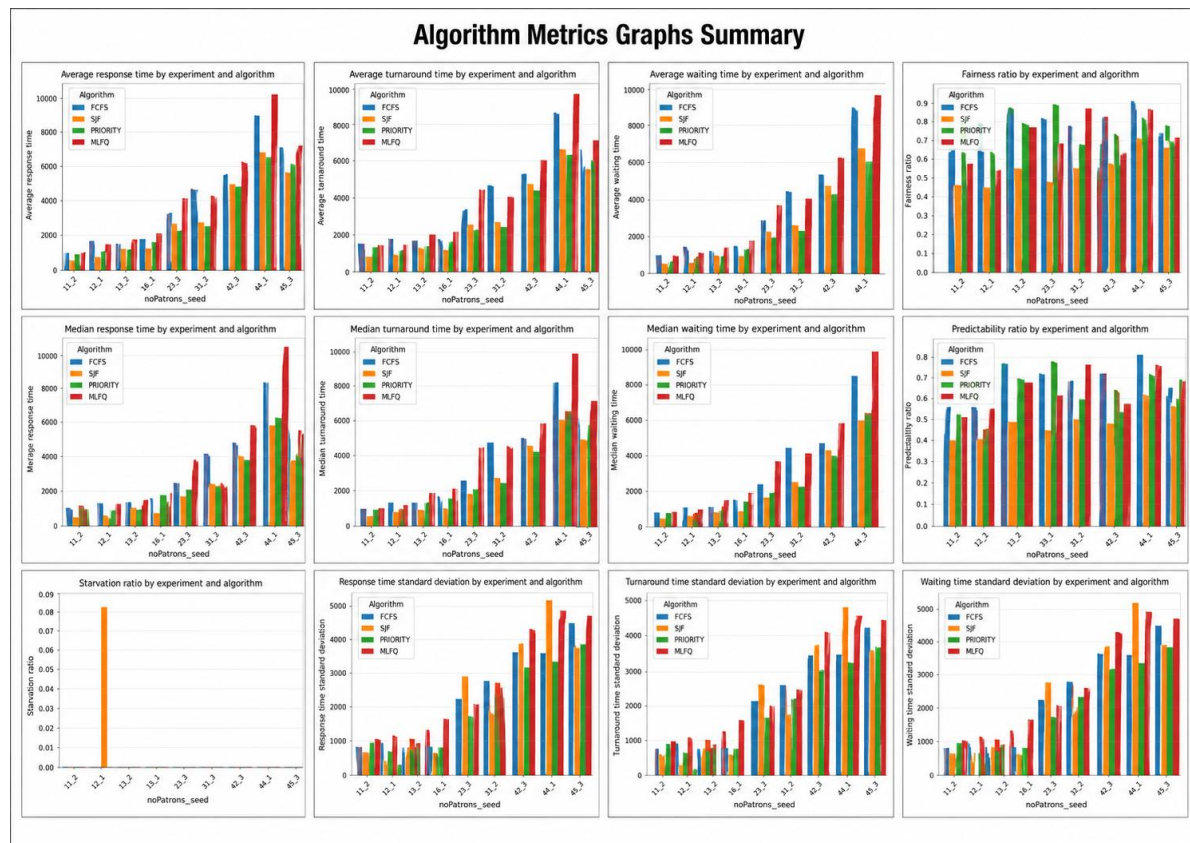


Figure 1: Algorithm Metrics Graphs

Conclusions

For a Bar, priority scheduling will be a smart way to go. Performs generally well across metrics. Provides better average waiting, response, and turnaround time, keeping customers happy. Is more consistent and predictable, good for bar reputation. Not exactly fair and might starve other customers, but no bar is perfect. There are always inevitable disadvantages.

Statement on AI Usage

I used generative AI (ChatGPT) for limited coding assistance, mainly to help debug implementation issues, structure Python helper scripts, and improve the clarity of metric computation and graph-generation code. The AI was used as a support tool for explanation, debugging, and code organisation, not as a replacement for running the experiments. All reported outputs, graphs, and conclusions are based on actual experiment runs from the scheduling simulation, and no artificial or fabricated results were included.

Formula Sheet

Let:

- n = number of patrons
- $W = [W_1, W_2, \dots, W_n]$ represent the total waiting time for patron i .
- $R = [R_1, R_2, \dots, R_n]$ represent the total response time for patron i .
- $T = [T_1, T_2, \dots, T_n]$ represent the total turnaround time for patron i .
- $D = [D_1, D_2, \dots, D_n]$ represent the number of drinks ordered by patron i .
- $C = [C_1, C_2, \dots, C_n]$ represent the last order completion time for patron i .

Formulas:

$$avgWaitingTime = \frac{\sum W_i}{n}$$

$$stdWaitingTime = \sqrt{(\sum (W_i - mean(W))^2) / n} \quad \# \text{ Similar for respond and turnaround times.}$$

$$fairnessRatio = \frac{(\sum W_i)^2}{(n \times \sum (W_i)^2)}$$

$$coefficientOfVariance = \frac{std(W)}{mean(W)}$$

$$predictabilityRatio = \frac{1}{(1 + coefficientOfVariance)}$$

$$starvationThreshold = Q3 + (1.5 \times IQR)$$

$$starvationCandidates = count(W_i > threshold)$$

$$starvationPossibility = \frac{starvationCandidates}{n}$$